

Mode 2: The "Speed Demon"

ESP-NOW Gateway (The Brains)

If you chose the ESP-NOW route, I respect you. You are part of the elite group of makers who refuse to let router lag delay their lights turning on.

 **IMPORTANT: YOU NEED A SECOND ESP FOR THIS!**  Because the ESP hidden inside your 3D-printed relay box (the Receiver) isn't connected to WiFi, it needs a commander. This code goes onto a **second, separate ESP board**—The Gateway.

The Gateway sits somewhere in your house, connects to your router, talks to Arduino Cloud (and Google Home / Alexa), and translates those commands into lightning-fast radio signals (ESP-NOW) sent straight to your relay box.

What you need:

- **A Second ESP32** (The Gateway). It doesn't need a fancy box, just a USB cable for power.
- The **MAC Address** you copied from your Receiver unit earlier.

Setup Instructions

Step 1: The Arduino Cloud Prep

1. Go to **Arduino IoT Cloud** and create a new "Thing" (This is your Gateway).
2. Add a variable named `switchESP` (Type: **CloudSwitch**, Permission: **Read & Write**, Policy: **On Change**).
3. Connect your Gateway ESP32 and set up your WiFi network in the "Secret" tab.
4. In WIFI documentation you can read more about how to set up the device.

Step 2: The MAC Address Magic

1. Open the Gateway code in the Arduino Cloud editor.
2. Find this line near the top: `uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};`
3. **Change those numbers** to the MAC address of your Receiver box. If you skip this, your Gateway will be shouting commands into the void!

Step 3: Upload and Dominate

1. Flash the code to your Gateway ESP.
2. Link your Arduino Cloud account to Google Home or Alexa (if you haven't already).
3. Say, *"Hey Google, turn on the switch."* The Gateway receives the command, fires an ESP-NOW packet, and your relay clicks instantly!

✚ Bonus: How to Add MORE Devices! (Expanding the Fleet)

Built a second relay box? Want to control the fan AND the lights? ESP-NOW is perfect for this. Here is how you modify the Gateway code to command multiple boxes:

1. Get the new MAC Address: Flash the Receiver code to your new box and copy its MAC address. **2. Add a new Cloud Variable:** Go to Arduino Cloud and add a second variable (e.g., `switchESP2`). **3. Update the Gateway Code:**

Add the new MAC:

C++

```
uint8_t broadcastAddress1[] = {0x84, 0x1F, ...}; // Box 1
uint8_t broadcastAddress2[] = {0xAA, 0xBB, ...}; // Box 2
```

-
- **Register the new Peer:** Inside `initESPNow()`, copy and paste the `peerInfo` block, but change it to use `broadcastAddress2`.
- **Add a new Send Function:** Create a `sendCommand2(bool state)` function that sends data specifically to `broadcastAddress2`.
- **Add the Cloud Trigger:** Arduino Cloud will generate a new function at the bottom of your code called `onswitchESP2Change()`. Inside it, just call `sendCommand2(switchESP2);!`

You can also ask AI how to do it 😊.

Now you have a central Gateway commanding an entire army of relay boxes around your house without clogging up your WiFi router! 🛡️🔊

● CODE

```
#include "thingProperties.h"
#include <esp_now.h>
#include <WiFi.h>

// --- IMPORTANT ---
// Your Wi-Fi credentials (SSID and Password) are automatically fetched
// from the "Secret" tab in the Arduino Cloud.
// The lines below are just a reminder; Arduino Cloud handles the connection.
// Make sure you enter them correctly in the "Secret" tab!
// e.g., SSID: Your_WiFi_Name | PASS: Your_Password

// ⚠ RECEIVER MAC ADDRESS ⚠
// Replace this with the MAC address you copied from the Receiver (Relay) unit!
uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
```

```

typedef struct struct_message {
    bool state;
} struct_message;

struct_message myData;
esp_now_peer_info_t peerInfo;
bool espNowInitialized = false;

// Callback function executed after sending data via ESP-NOW
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
    // Optional status logs (Uncomment for debugging)
    // Serial.print("Last Packet Send Status: ");
    // Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" : "Delivery
Fail");
}

void sendCommand(bool state) {
    // Don't try to send if ESP-NOW isn't ready
    if (!espNowInitialized) return;

    myData.state = state;
    esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *) &myData, sizeof(myData));

    if (result == ESP_OK) {
        Serial.println("Sent with success");
    } else {
        Serial.println("Error sending the data");
    }
}

void initESPNow() {
    // We must check if Wi-Fi is connected first so we can grab its channel
    if (WiFi.status() != WL_CONNECTED) return;

    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        return;
    }

    esp_now_register_send_cb(OnDataSent);

    // Peer Configuration (Registering the Receiver)
    memcpy(peerInfo.peer_addr, broadcastAddress, 6);
    peerInfo.channel = 0; // Use the same channel as the current Wi-Fi connection
    peerInfo.encrypt = false;

    if (esp_now_add_peer(&peerInfo) != ESP_OK) {

```

```

    Serial.println("Failed to add peer");
  } else {
    espNowInitialized = true;
    Serial.print("ESP-NOW initialized on channel: ");
    Serial.println(WiFi.channel());
  }
}

void setup() {
  Serial.begin(9600);
  delay(1500);

  // Initialization defined in thingProperties.h
  initProperties();

  // Connect to Arduino IoT Cloud (uses data from the Secret tab)
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}

void loop() {
  ArduinoCloud.update();

  // We only try to initialize ESP-NOW AFTER successfully connecting to Wi-Fi
  // This ensures we are on the correct Wi-Fi channel for ESP-NOW to work
  if (ArduinoCloud.connected() && !espNowInitialized) {
    initESPNow();
  }
}

// --- CLOUD VARIABLE CALLBACK ---
// This function name must perfectly match the one generated in thingProperties.h
// It runs whenever you flip the switch in the Cloud / Alexa / Google Home
void onswitchESPChange() {
  Serial.print("Cloud variable switchESP changed to: ");
  if(switchESP) {
    Serial.println("ON");
  } else {
    Serial.println("OFF");
  }
}

// Send the new state to the Receiver box via ESP-NOW
sendCommand(switchESP);
}

```